The AKS Algorithm

Siddharth Narayan Mishra

March 26, 2026

The Question

Is primality testing in **P**?

- Fermat: fast but unreliable
- Miller-Rabin: fast, probabilistic

AKS (2002)

PRIMES is in P

What AKS Achieves

- Deterministic
- Unconditional
- Polynomial time

$$\text{Time: } \mathcal{O}((\log n)^c)$$

Significance

First proof that primality testing is efficiently solvable

The Core Idea

Key Identity

n is prime iff:

$$(X + a)^n \equiv X^n + a \pmod{n}$$

- Holds for all a with $\gcd(a, n) = 1$
- Comes from binomial expansion

Proof ($\Rightarrow$ direction)

Assume n is prime.

$$(X+a)^n = \sum_{i=0}^{n} \binom{n}{i} a^{n-i} X^i$$

For $0 < i < n$:

$$\binom{n}{i} = \frac{n!}{i!(n-i)!}$$

$$\binom{n}{i} \equiv 0 \pmod{n}$$

Proof (continued)

Thus:

$$(X + a)^n \equiv X^n + a^n \pmod{n}$$

By Fermat's Little Theorem:

$$a^n \equiv a \pmod{n}$$

$$\Rightarrow (X + a)^n \equiv X^n + a \pmod{n}$$

Proof ($\Leftarrow$ direction)

Assume:

$$(X + a)^n \equiv X^n + a \pmod{n}$$

Suppose n is composite.
Let $q^k \parallel n$ (largest power of q dividing n).

Proof (continued)

Let $q^k \parallel n$ (i.e., $q^k \mid n$ but $q^{k+1} \nmid n$).

Consider:

$$\binom{n}{q} = \frac{n(n-1)\cdots(n-q+1)}{q!}$$

Rewrite:

$$\binom{n}{q} = \frac{q^k(n-1)\cdots(n-q+1)}{q\cdot(q-1)!} = \frac{q^{k-1}(n-1)\cdots(n-q+1)}{(q-1)!}$$

Key point:

- Numerator contains exactly q^{k-1} as the highest power of q
- None of $(n-1), \ldots, (n-q+1)$ are divisible by q
- Denominator $(q-1)!$ contains no factor of q

So $\binom{n}{q}$ contains exactly q^{k-1} as a factor.

But n contains q^k as a factor.

$$n \nmid \binom{n}{q} \quad \Rightarrow \quad \binom{n}{q} \not\equiv 0 \pmod{n}$$

Example: Prime Case ($n = 5$)

$$(X+1)^5 = X^5 + 5X^4 + 10X^3 + 10X^2 + 5X + 1$$

Modulo 5:

$$(X+1)^5 \equiv X^5 + 1 \pmod 5$$

Observation

All middle coefficients vanish modulo 5

Pascal Triangle Insight

Row 5 of Pascal's Triangle:

$$
\begin{array}{c}
1 \\
1\ 1 \\
1\ 2\ 1 \\
1\ 3\ 3\ 1 \\
1\ 4\ 6\ 4\ 1 \\
1\ 5\ 10\ 10\ 5\ 1
\end{array}
$$

- For prime p, all interior entries are divisible by p
- Only the endpoints are 1

Example: Composite Case ($n = 4$)

$$(X+1)^4 = X^4 + 4X^3 + 6X^2 + 4X + 1$$

Modulo 4:

$$(X+1)^4 \equiv X^4 + 2X^2 + 1 \pmod 4$$

Observation

Middle coefficient $6 \equiv 2 \pmod 4$ does not vanish

$$\Rightarrow \text{Identity fails}$$

Why Not Use This Directly?

- The identity gives a **perfect characterization** of primes

$$(X + a)^n = \sum_{i=0}^{n} \binom{n}{i} a^{n-i} X^i$$

- Polynomial has $n + 1$ terms
- Computing it requires $\Omega(n)$ operations

Problem

Input size is $\log n$, but runtime is $\Omega(n)$

Conclusion

Exponential time $\Rightarrow$ not efficient

Reducing the Cost

Work modulo:

$$X^r - 1$$

- $X^r \equiv 1$
- Degrees wrap around

Effect

Polynomial size reduces from n to r

Example: Working Modulo $X^r - 1$

Let $r = 5$. Then:

$$X^5 \equiv 1 \pmod{X^5 - 1}$$

Example: Reduce X^7

$$X^7 = X^{5+2} = X^5 \cdot X^2 \equiv 1 \cdot X^2 = X^2$$

Example: Reduce a polynomial

$$f(X) = X^7 + 3X^6 + 2X^2$$

$$X^7 \equiv X^2, \quad X^6 = X^{5+1} \equiv X$$

$$\Rightarrow f(X) \equiv X^2 + 3X + 2X^2 = 3X^2 + 3X$$

Key Idea

Every time degree $\geq r$, replace X^r by 1 and reduce

New Test

Check:

$$(X + a)^n \equiv X^n + a \pmod{X^r - 1,\ n}$$

- Much faster
- Works for primes

Issue

Some composites may now pass

Fixing False Positives

- Work modulo $X^r - 1$ (compression)
- Test multiple values of a

Key Condition

$$\mathrm{ord}_r(n) > (\log n)^2$$

- Ensures composite numbers cannot satisfy identity for many a

What is $\mathrm{ord}_r(n)$?

Definition

$$\mathrm{ord}_r(n) = \min\{k > 0 \mid n^k \equiv 1 \pmod{r}\}$$

- Measures how n behaves modulo r
- Captures the **cycle length** of powers of n

Example

$$2^3 \equiv 1 \pmod{7} \Rightarrow \mathrm{ord}_7(2) = 3$$

Why Do We Need Large Order?

- We want:

$$\mathrm{ord}_r(n) > (\log n)^2$$

Reason

Ensures n does not behave "periodically" too early modulo r

- Prevents composite numbers from mimicking primes
- Forces large algebraic structure in the proof

Existence of r

Lemma

There exists $r \leq (\log n)^5$ such that:

$$\mathrm{ord}_r(n) > (\log n)^2$$

Proof.

Let:

$$B = \lceil (\log n)^5 \rceil$$

Assume for contradiction that for all $r \leq B$:

$$\mathrm{ord}_r(n) \leq (\log n)^2$$

Key Construction

Then for every $r \leq B$, there exists $k \leq (\log n)^2$ such that:

$$n^k \equiv 1 \pmod{r}$$

$$\Rightarrow r \mid (n^k - 1)$$

Thus every $r \leq B$ divides:

$$N = \prod_{k=1}^{\lfloor (\log n)^2 \rfloor} (n^k - 1)$$

Bounding N

Each term:

$$n^k - 1 < n^k$$

So:

$$N < \prod_{k=1}^{(\log n)^2} n^k = n^{\sum_{k=1}^{(\log n)^2} k}$$

$$= n^{\frac{(\log n)^2((\log n)^2+1)}{2}} \leq n^{(\log n)^4}$$

$$= 2^{(\log n)^5}$$

Contradiction via LCM

But if every $r \leq B$ divides N, then:

$$\mathrm{lcm}(1, 2, \ldots, B) \mid N$$

Fact

$$\mathrm{lcm}(1, 2, \ldots, B) \geq 2^B$$

Thus:

$$2^B \leq N < 2^{(\log n)^5}$$

Final Contradiction

Recall:

$$B = (\log n)^5$$

So:

$$2^B = 2^{(\log n)^5}$$

But we showed:

$$N < 2^{(\log n)^5}$$

$$\Rightarrow 2^B \nmid N$$

Contradiction

Not all $r \leq B$ can divide N

Conclusion

- There exists some $r \leq (\log n)^5$ such that:

$$\mathrm{ord}_r(n) > (\log n)^2$$

AKS Algorithm

1. Check if $n = a^b$ for some $b > 1$
2. Find smallest r with $\mathrm{ord}_r(n) > (\log n)^2$
3. If $\gcd(a, n) > 1$ for some $a \leq r$, output composite
4. If $n \leq r$, output prime
5. For $a = 1$ to $O(\sqrt{r} \log n)$:

$$(X + a)^n \equiv X^n + a \pmod{X^r - 1, n}$$

6. Output prime

Why Each Step Matters

- Step 1: removes perfect powers
- Step 2: ensures strong algebraic structure
- Step 3: detects small factors quickly
- Step 4: handles small cases
- Step 5: main polynomial test

Key Point

All steps together eliminate every possible composite

Correctness: Setup

Assume the algorithm reaches the final step and outputs PRIME.

Then:

- n is not a perfect power
- $\gcd(a, n) = 1$ for all $a \leq r$
- $\mathrm{ord}_r(n) > (\log n)^2$
- For all $1 \leq a \leq \ell$:

$$(X + a)^n \equiv X^n + a \pmod{X^r - 1,\ n}$$

Let p be any prime divisor of n.

Reducing Modulo p

Since $p \mid n$:

$$(X + a)^n \equiv X^n + a \pmod{X^r - 1,\ p}$$

Also, since p is prime:

$$(X + a)^p \equiv X^p + a \pmod{p}$$

Raise both sides to power n/p:

$$(X + a)^n \equiv (X^p + a)^{n/p}$$

Key Identity

Combining:

$$(X^p + a)^{n/p} \equiv X^n + a \pmod{X^r - 1,\ p}$$

Let $Y = X^p$:

$$(Y + a)^{n/p} \equiv Y^{n/p} + a$$

Observation

n/p satisfies the same identity as a prime

Introspective Property

Define:

$$f(X)^m \equiv f(X^m) \pmod{X^r - 1,\ p}$$

Then:

- p satisfies this for $f(X) = X + a$
- n/p also satisfies it

Closure

Products of such numbers also satisfy this property

Building a Large Set

Define:

$$I = \left\{ \left(\frac{n}{p}\right)^i \cdot p^j \;\middle|\; i, j \geq 0 \right\}$$

All elements of I satisfy:

$$f(X)^m \equiv f(X^m)$$

Reduce modulo r:

$$G = \{m \bmod r \mid m \in I\}$$

$$|G| > (\log n)^2$$

I Contains All Powers of n

Recall $n = p \cdot \frac{n}{p}$, so setting $i = j$ gives:

$$\left(\frac{n}{p}\right)^i \cdot p^i \;=\; n^i \;\in\; I \qquad \text{for all } i \geq 0.$$

In particular, I contains:

$$n^0,\; n^1,\; n^2,\; \ldots$$

Defining G

We reduce every element of I modulo r:

$$G = \{ m \bmod r \mid m \in I \}$$

Think of G as the set of "fingerprints" that elements of I leave when divided by r.

Since I contains all powers of n, G contains in particular:

$$n^0 \bmod r, \quad n^1 \bmod r, \quad n^2 \bmod r, \quad \ldots$$

The question is: how many of these are distinct?

How Many Distinct Powers of n Are There mod r?

The powers $n^0, n^1, n^2, \ldots$ eventually repeat modulo r.

The number of distinct values among $\{n^i \bmod r\}$ is exactly $\mathrm{ord}_r(n)$ — the smallest positive k such that $n^k \equiv 1 \pmod{r}$.

Specifically, the values:

$$n^0 \bmod r, \quad n^1 \bmod r, \quad \ldots, \quad n^{\mathrm{ord}_r(n)-1} \bmod r$$

are all distinct, and after that they repeat.

Conclusion: $|G| > (\log n)^2$

Since all powers $n^0, n^1, \ldots, n^{\mathrm{ord}_r(n)-1}$ are distinct modulo r, and all of them land in G:

$$|G| \;\geq\; \mathrm{ord}_r(n)$$

We chose r in Step 2 of AKS precisely so that:

$$\mathrm{ord}_r(n) \;>\; (\log n)^2$$

Chaining the two inequalities:

$$\boxed{|G| \;\geq\; \mathrm{ord}_r(n) \;>\; (\log n)^2}$$

Working Modulo a Prime: $\mathbb{F}_r$

Throughout the proof we work with arithmetic modulo r, where r is prime. This means:

$$0, 1, 2, \ldots, r-1$$

are the only numbers, and we always reduce modulo r.
For example, with $r = 5$: $\quad 3 + 4 = 2, \quad 3 \times 4 = 2.$
We call this number system $\mathbb{F}_r$.

Polynomials mod r

We can also consider polynomials whose coefficients live in $\mathbb{F}_r$:

$$X^3 + 2X + 4, \qquad X^2 + 1, \qquad \ldots$$

with all coefficients reduced modulo r.
For example, with $r = 5$:

$$(2X + 3) + (X + 4) = 3X + 7 = 3X + 2$$

We call this collection $\mathbb{F}_r[X]$.

Reducing Polynomials Further: Mod $h(X)$

Just as we reduce numbers modulo r, we can reduce *polynomials* modulo another polynomial $h(X)$.
Every time $h(X)$ appears as a factor, we set it to zero, exactly like setting $r = 0$ in modular arithmetic.

Example

If $h(X) = X^2 + 1$ and $r = 5$:

$$X^3 \bmod (X^2 + 1) = X \cdot X^2 = X \cdot (-1) = -X = 4X$$

since $X^2 \equiv -1 \pmod{h(X)}$.

What Is a Multiplicative Inverse?

The multiplicative inverse of a number a is the number b such that:

$$a \times b = 1$$

For example, in ordinary arithmetic:

$$\text{the inverse of 3 is } \frac{1}{3}, \quad \text{since } 3 \times \frac{1}{3} = 1.$$

In $\mathbb{F}_5$, fractions do not exist, but inverses still do:

$$\text{the inverse of 3 is 2}, \quad \text{since } 3 \times 2 = 6 \equiv 1 \pmod 5.$$

Irreducible Polynomials

A polynomial $h(X)$ over $\mathbb{F}_r$ is called *irreducible* if it cannot be factored into two smaller polynomials, the analogue of a prime number, but for polynomials.

Example

Over $\mathbb{F}_2$: $X^2 + X + 1$ is irreducible (it has no roots: $0^2 + 0 + 1 = 1 \neq 0$, $1^2 + 1 + 1 = 1 \neq 0$).

When we reduce modulo an irreducible $h(X)$, every nonzero polynomial has a multiplicative inverse, we can always divide by any nonzero element. We call such a number system a *field*.

The Field $\mathbb{F} = \mathbb{F}_r[X]/(h(X))$

Putting it all together:

$$\mathbb{F} = \mathbb{F}_r[X]/(h(X))$$

is the collection of all polynomials with coefficients in $\mathbb{F}_r$, reduced modulo $h(X)$.

It behaves like a number system where:

- coefficients are reduced mod r,
- polynomial degree is kept below $\deg h(X)$,
- every nonzero element has an inverse (since h is irreducible).

This is the arena in which we will count elements of $\hat{G}$.

What Is $\hat{G}$?

Fix an irreducible factor $h(X)$ of $X^r - 1$ over $\mathbb{F}_p$.

$$P = \left\{ \prod_{a=0}^{\ell} (X+a)^{e_a} \;\middle|\; e_a \geq 0, \sum_{a=0}^{\ell} e_a < t \right\}$$

where $\ell = \lfloor \sqrt{\varphi(r)} \log n \rfloor$.
$\hat{G}$ is the set of *distinct* images of polynomials in P, inside the field $\mathbb{F} = \mathbb{F}_p[X]/(h(X))$:

$$\hat{G} = \{ f(X) \bmod h(X) \mid f(X) \in P \}$$

Why Distinct Polynomials Stay Distinct in $\hat{G}$

Claim

If $f(X)$ and $g(X)$ are two distinct polynomials in P of degree $< t = |G|$, then they map to *different* elements in $\hat{G}$.

Suppose not: $f(X) = g(X)$ inside $\mathbb{F}$.
Then for any introspective $m \in I$, applying the introspection property to both sides:

$$f(X^m) = [f(X)]^m = [g(X)]^m = g(X^m) \quad \text{in } \mathbb{F}$$

So $f(X^m) - g(X^m) = 0$ in $\mathbb{F}$, meaning X^m is a root of the polynomial $Q(Y) = f(Y) - g(Y)$.

Reaching a Contradiction

As m ranges over all elements of G, the values X^m are all *distinct* in $\mathbb{F}$, because X is a primitive r-th root of unity in $\mathbb{F}$.

So $Q(Y) = f(Y) - g(Y)$ has at least $|G| = t$ distinct roots in $\mathbb{F}$.

But f and g both have degree $< t$, so:

$$\deg Q \ < \ t$$

A polynomial of degree $< t$ cannot have t distinct roots in a field.

Contradiction

Therefore $f(X) \neq g(X)$ in $\mathbb{F}$. $\square$

How Many Distinct Polynomials of Degree $< t$ Are in P?

We need to count products $\prod_{a=0}^{\ell}(X+a)^{e_a}$ with $e_a \geq 0$ and total degree:

$$\sum_{a=0}^{\ell} e_a \;\leq\; t-1$$

This is a standard *stars and bars* count: distributing at most $t-1$ among $\ell+1$ values.
The number of ways is:

$$\binom{t-1+\ell+1}{\ell+1} = \binom{t+\ell}{\ell+1}$$

So $|\hat{G}| \geq \binom{t+\ell}{\ell+1}$

Since each such polynomial is distinct in $\hat{G}$:

$$|\hat{G}| \;\geq\; \binom{t+\ell}{\ell+1}$$

Now we use two facts:

- $t = |G| > \log^2 n$, so $\sqrt{t} > \log n$
- $\varphi(r) \geq t$, so $\ell = \lfloor \sqrt{\varphi(r)} \log n \rfloor \geq \lfloor \sqrt{t} \log n \rfloor$

Write $m = \lfloor \sqrt{t} \log n \rfloor \geq 2$ (since $n \geq 4$). Then $\ell \geq m$, so:

$$\binom{t+\ell}{\ell+1} \;\geq\; \binom{2m+1}{m+1} \;=\; \binom{2m+1}{m}$$

A Simple Binomial Inequality

Claim

For any integer $m \geq 2$: $\binom{2m+1}{m} > 2^{m+1}$

Proof.

The sum of all binomial coefficients gives 2^{2m+1}. The middle term $\binom{2m+1}{m}$ is the largest, and there are $m+1$ terms from $\binom{2m+1}{m}$ up to $\binom{2m+1}{2m+1}$, so:

$$(m+1)\binom{2m+1}{m} \geq 2^{2m} \implies \binom{2m+1}{m} \geq \frac{2^{2m}}{m+1} > 2^{m+1}$$

for $m \geq 2$. $\square$

Completing the Lower Bound

Applying the inequality with $m = \lfloor \sqrt{t} \log n \rfloor$:

$$|\hat{G}| \;\geq\; \binom{2m+1}{m} \;>\; 2^{m+1} \;>\; 2^{\sqrt{t}\log n} \;=\; n^{\sqrt{t}}$$

Since $t = |G|$:

Lower Bound

$$|\hat{G}| \;>\; n^{\sqrt{|G|}}$$

Finding a Polynomial That $\hat{G}$ Satisfies

Consider the subset of I:

$$J = \left\{ \left(\frac{n}{p}\right)^i \cdot p^j \;\middle|\; 0 \le i, j \le \lfloor \sqrt{t} \rfloor \right\}$$

where $t = |G|$.
J has $(\lfloor \sqrt{t} \rfloor + 1)^2 > t$ elements.
But G has only t distinct residues mod r.
By pigeonhole, two distinct elements of J must give the same residue mod r:

$$m_1 \equiv m_2 \pmod{r}, \qquad m_1 \neq m_2, \quad m_1, m_2 \in J.$$

What Happens When n Is Not a Power of p

The elements of J have the form $\left(\frac{n}{p}\right)^i p^j$.
If two such elements are equal (not just mod r, but as integers):

$$\left(\frac{n}{p}\right)^{i_1} p^{j_1} = \left(\frac{n}{p}\right)^{i_2} p^{j_2}$$

then rearranging gives $n^{i_1-i_2} = p^{j_2-j_1-i_2+i_1}$, meaning n is a power of p.
So if n is *not* a power of p, all elements of J are *distinct as integers*, giving us genuinely distinct $m_1 \neq m_2$ with $m_1 \equiv m_2 \pmod{r}$.

A Polynomial Satisfied by All of $\hat{G}$

Since $m_1 \equiv m_2 \pmod{r}$, we have $X^{m_1} = X^{m_2}$ in $\mathbb{F}$ (because $X^r = 1$ in $\mathbb{F}$). So for any $f(X) \in P$, the introspection property gives:

$$[f(X)]^{m_1} = f(X^{m_1}) = f(X^{m_2}) = [f(X)]^{m_2} \quad \text{in } \mathbb{F}$$

Every element $f(X) \in \hat{G}$ is therefore a root of:

$$Q(Y) = Y^{m_1} - Y^{m_2}$$

Bounding the Degree of $Q(Y)$

The degree of $Q(Y) = Y^{m_1} - Y^{m_2}$ is $\max(m_1, m_2)$.
Both m_1 and m_2 lie in J, so:

$$m_1, m_2 \;\leq\; \left(\frac{n}{p}\right)^{\lfloor\sqrt{t}\rfloor} \cdot p^{\lfloor\sqrt{t}\rfloor} \;=\; n^{\lfloor\sqrt{t}\rfloor} \;\leq\; n^{\sqrt{t}}$$

Therefore:

$$\deg Q \;\leq\; n^{\sqrt{t}}$$

Since a polynomial of degree d has at most d roots in a field, and all $|\hat{G}|$ elements are roots of Q:

$$\boxed{|\hat{G}| \;\leq\; n^{\sqrt{|G|}}}$$

The Contradiction

We now have two bounds on $|\hat{G}|$ simultaneously:

Lower Bound

$$|\hat{G}| > n^{\sqrt{|G|}}$$

(always true)

Upper Bound

$$|\hat{G}| \leq n^{\sqrt{|G|}}$$

(if n not a power of p)

These cannot both hold. So our assumption was wrong:

$$n \text{ must be a power of } p.$$

Conclusion

We have shown $n = p^k$ for some $k \geq 1$.
But AKS checks at the very first step whether n is a perfect power, and outputs COMPOSITE if so.
Since n passed that check, we must have $k = 1$.

Final Result

$$n = p \implies n \text{ is prime.} \qquad \blacksquare$$

Time Complexity: Overview

Let $k = \log n$ be the input size (number of bits in n).
The AKS algorithm has five steps. We analyze each one and show every step runs in time polynomial in k.

Goal

Total time $= \mathcal{O}((\log n)^c)$ for some fixed constant c.

Step 1: Perfect Power Check

Check whether $n = a^b$ for some $a \in \mathbb{N}$, $b > 1$.
We try all exponents $b \leq \log n$, and for each b, compute the integer b-th root of n using binary search.
Each root computation on a $\log n$-bit number costs $\mathcal{O}((\log n)^2)$, and we do $\mathcal{O}(\log n)$ of them:

$$\text{Step 1 time} = \mathcal{O}((\log n)^3)$$

Step 2: Finding r

Find the smallest r such that $\mathrm{ord}_r(n) > (\log n)^2$.
By Lemma 4, such an r exists with $r \leq \log^5 n$. We try each candidate r and check whether $n^i \equiv 1 \pmod{r}$ for all $i \leq (\log n)^2$, stopping as soon as one fails.
There are $\mathcal{O}(\log^5 n)$ candidates, each requiring $\mathcal{O}((\log n)^2)$ modular exponentiations costing $\mathcal{O}(\log n)$ each:

$$\text{Step 2 time} = \mathcal{O}((\log n)^5 \cdot (\log n)^2) = \mathcal{O}((\log n)^7)$$

Step 3: GCD Checks

For each a with $1 \leq a \leq r$, compute $\gcd(a, n)$.
If any gcd lies strictly between 1 and n, output COMPOSITE.
Each GCD via the Euclidean algorithm costs $\mathcal{O}(\log n)$, and we do at most $r = \mathcal{O}(\log^5 n)$ of them:

$$\text{Step 3 time} = \mathcal{O}((\log n)^5 \cdot \log n) = \mathcal{O}((\log n)^6)$$

Step 4: Small n Check

If $n \leq r$, output $\textsc{Prime}$ directly.
This is a single comparison and costs $\mathcal{O}(1)$, negligible.
If we reach Step 5, we know $n > r$, so n is large enough that the polynomial checks are meaningful.

Step 5: The Main Loop

For each a from 1 to $\ell = \lfloor \sqrt{\varphi(r)} \log n \rfloor$, verify:

$$(X + a)^n \equiv X^n + a \pmod{X^r - 1,\ n}$$

Since $\varphi(r) \leq r = \mathcal{O}(\log^5 n)$:

$$\ell = \mathcal{O}\left(\sqrt{\log^5 n} \cdot \log n\right) = \mathcal{O}((\log n)^{7/2})$$

This is the most expensive step, we analyze its cost next.

Cost of Each Polynomial Exponentiation

Each check requires computing $(X + a)^n \bmod (X^r - 1, n)$.
We use repeated squaring, which takes $\mathcal{O}(\log n)$ multiplications.
Each multiplication involves polynomials of degree $< r$ with coefficients of size $\log n$, costing $\mathcal{O}(r \log n)$.
So the cost per value of a is:

$$\mathcal{O}(\log n) \times \mathcal{O}(r \log n) = \mathcal{O}(r(\log n)^2)$$

Total Time Complexity

Putting all steps together, Step 5 dominates:

$$\text{Step 5 time} = \ell \cdot \mathcal{O}(r(\log n)^2)$$

Substituting $\ell = \mathcal{O}((\log n)^{7/2})$ and $r = \mathcal{O}((\log n)^5)$:

$$= \mathcal{O}\Big((\log n)^{7/2} \cdot (\log n)^5 \cdot (\log n)^2\Big) = \mathcal{O}\Big((\log n)^{21/2}\Big)$$

Conclusion

AKS runs in time $\mathcal{O}((\log n)^{21/2})$, which is polynomial in the input size $k = \log n$. Hence $\textsc{Primes} \in \mathbf{P}$.

Can We Do Better?

The complexity $\mathcal{O}((\log n)^{21/2})$ is polynomial, but can it be improved?
The bottleneck is Step 2: finding r such that $\mathrm{ord}_r(n) > (\log n)^2$. A tighter bound on r directly reduces the overall complexity.
Lenstra and Pomerance (2011) showed that a better choice of r and a more careful analysis brings the complexity down to:

$$\mathcal{O}((\log n)^6)$$

Further improvements are possible assuming standard conjectures in number theory, such as the Artin conjecture on primitive roots.

How Does AKS Compare in Practice?

Algorithm	**Deterministic?**	**Always correct?**	**Complexity**
Trial division	Yes	Yes	$\mathcal{O}(\sqrt{n})$
Fermat test	Yes	No	$\mathcal{O}((\log n)^2)$
Miller-Rabin	No	Yes (w.h.p.)	$\mathcal{O}((\log n)^3)$
AKS	Yes	Yes	$\mathcal{O}((\log n)^6)$

AKS is the only algorithm that is both deterministic and provably correct for all inputs, with polynomial runtime.

The Practical Catch

Despite its theoretical importance, AKS is rarely used in practice.
Miller-Rabin runs much faster and the probability of error can be made astronomically small by repeating the test a few times.
For a 2048-bit number, Miller-Rabin with 40 rounds has an error probability of at most $4^{-40} \approx 10^{-24}$, smaller than the probability of a hardware failure mid-computation.

The honest summary

AKS matters not because it is the fastest, but because it *settles a question*: can primality be decided without randomness?

What AKS Actually Proved

Before 2002, the best deterministic primality tests either:

- ran in superpolynomial time, or
- required unproven number-theoretic assumptions.

AKS gave the first unconditional, deterministic, polynomial-time algorithm, settling a question that had been open for decades.
It also demonstrated something deeper: the polynomial identities that characterize primes (Lemma 1) contain enough information to *decide* primality efficiently, without any randomness.

Where Does AKS Sit in Complexity Theory?

Recall the major complexity classes:

$$\mathbf{P} \subseteq \mathbf{BPP} \subseteq \cdots$$

BPP is the class of problems solvable in polynomial time with a randomized algorithm (bounded error probability).
Miller-Rabin places $\textsc{Primes}$ in **BPP**.
AKS places $\textsc{Primes}$ unconditionally in **P**:

$$\textsc{Primes} \in \mathbf{P}$$

This is one of the few natural problems known to lie in **P** after a long period of sitting only in **BPP**.

Takeaway

Randomness is not necessary.

The AKS algorithm shows that one of the most fundamental questions in mathematics, *is this number prime?*, can be answered efficiently, deterministically, and unconditionally.

Final Conclusion

$$\textsc{Primes} \in \mathbf{P}$$

Primality testing is not just tractable, it is *provably* tractable, with no randomness, no unproven assumptions, and no exceptions.